



Applicazioni Web

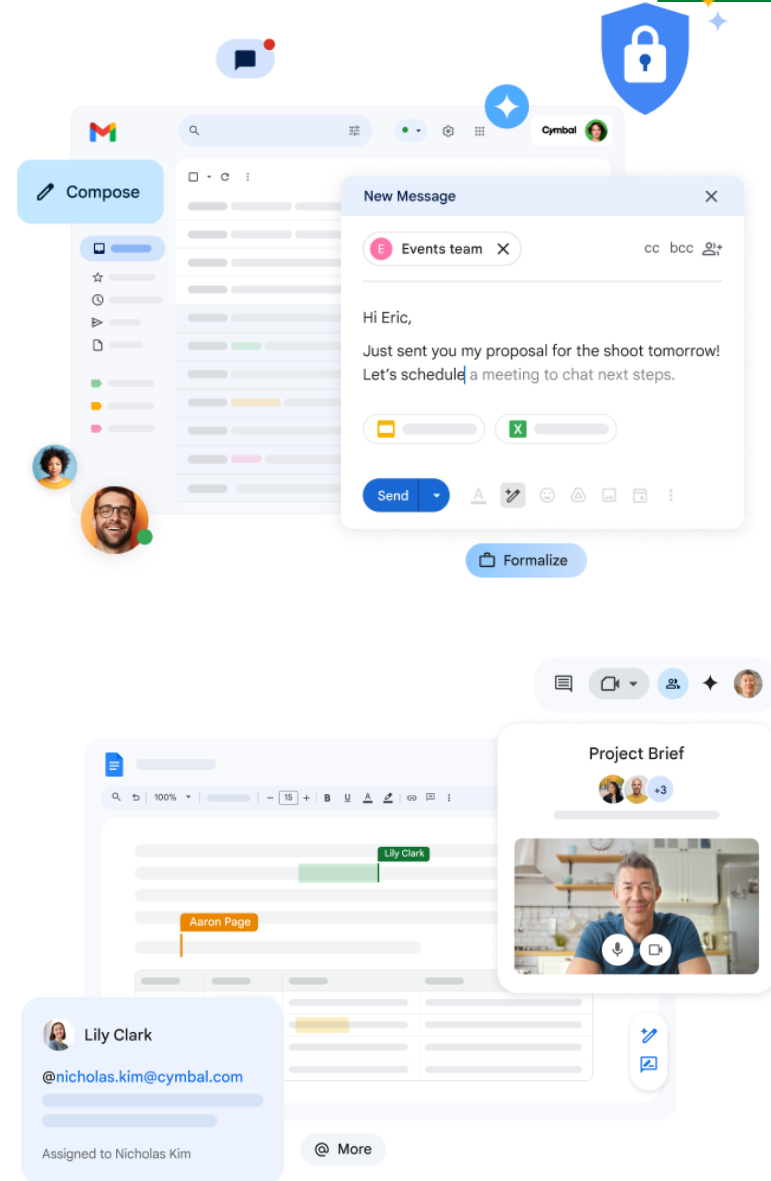


Cosa sono le Web Application?

- Applicazioni accessibili tramite **browser**
 - Estensioni delle pagine web
 - Non richiedono l'installazione di programmi aggiuntivi
 - Forniscono servizi dinamici grazie alla rete internet
- Nell'ambito sanitario, le web app sono fondamentali per:
 - Gestire dati sanitari in modo sicuro e flessibile
 - Accesso immediato a funzionalità come prenotazioni, referti, consulti online
 - Favorire l'integrazione tra sistemi diversi e la condivisione di informazioni
- Negli ultimi anni si tende a realizzare questo tipo di applicazioni

Esempio 1: Google Workspace

- Suite di strumenti di produttività e collaborazione basata sul cloud
 - docs, drive, mail, meet, etc.
- Aspetti chiave
 - Accesso Universale via browser
 - Collaborazione in tempo reale
 - Integrazione dei tool
- Nota:
 - I vostri dati sono nella rete (nel cloud)



Esempio 2: Trello



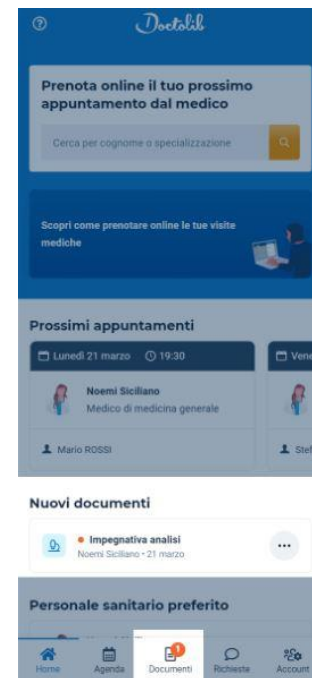
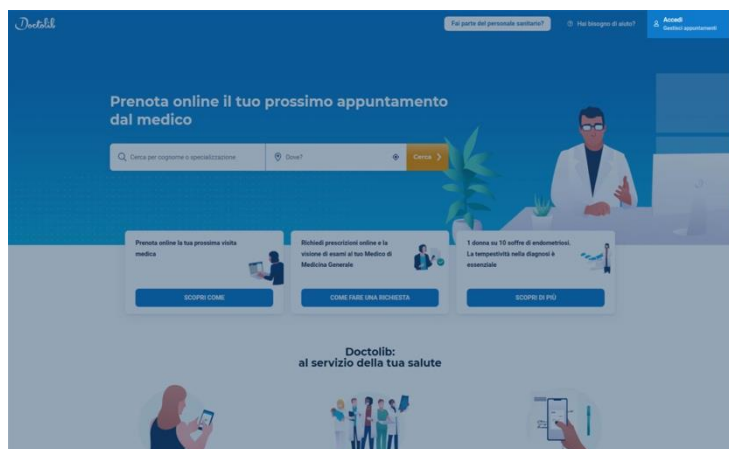
- Trello è una Web App per la gestione di progetti e attività.
- Basata su un sistema visivo di bacheche, liste e schede.
- Accessibile da qualsiasi browser, senza installazione.
- Consente la collaborazione in tempo reale tra più utenti.
- Integrabile con strumenti esterni come Google Drive, Slack, ecc.



Esempio 3: Doctolib



- Gestione di prenotazioni di visite, sessioni di telemedicina e la gestione degli appuntamenti
- Caratteristiche principali:
 - Prenotazione online di visite mediche specialistiche
 - Consultazioni video tramite telemedicina
 - Gestione agenda per medici e strutture
 - Notifiche e promemoria automatici per i pazienti
 - Integrazione con sistemi gestionali sanitari





Web Browser

- Ruolo del browser
 - È l'interfaccia che consente all'utente di interagire con la Web App
 - Interpreta il codice HTML, CSS, JavaScript proveniente dal server
 - Controlla l'aspetto grafico: colori, font, layout, animazioni
 - Interagisce con il server per rendere dinamica l'app
- Funziona come client universale
 - Non richiede installazione specifica per ogni app
 - Funziona su qualsiasi dispositivo connesso a Internet
- Limitazioni
 - Dipende dalla connessione Internet
 - Ha accesso limitato alle funzionalità hardware del dispositivo
 - Esperienza utente talvolta meno fluida rispetto a un'app installata

Tecnologie Web



HTML – la struttura

- Definisce gli elementi della pagina: testi, titoli, immagini, form



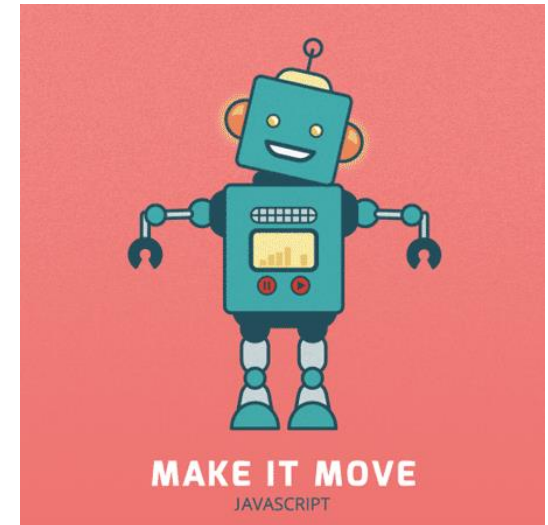
CSS – lo stile

- Controlla l'aspetto grafico: colori, font, layout, animazioni



JavaScript – il comportamento

- Rende la pagina interattiva: risposte ai clic, dinamiche, chiamate a server/API





Vantaggi webapp

- Scalabilità
 - Facile adattamento a un numero crescente di utenti
- Aggiornamenti Centralizzati
 - Nuove funzionalità e correzioni gestite lato server
- Sicurezza dei Dati
 - I dati sensibili restano su server protetti
- Flessibilità
 - Accesso da browser su dispositivi diversi (PC, tablet, smartphone)



Web app vs App Native

- Applicazioni Native:
 - Progettate per un determinato sistema operativo (Windows, Android).
 - Scaricate da uno store (Microsoft Store, Google Play) e installate sul dispositivo.
 - Aggiornamenti e manutenzione dipendono dall'utente.
- Applicazioni Web:
 - Funzionano direttamente nel browser.
 - Non richiedono installazione.
 - Immediatamente fruibili su qualsiasi dispositivo con browser compatibile.
 - Sviluppatori aggiornano un'unica base di codice (il server)



Dalle origini: il modello Client-Server e HTTP



Evoluzione delle Applicazioni web

-  In origine:
 - contenuti statici, nessuna interazione
-  Oggi – Web App:
 - Contenuti dinamici
 - Gestione dati in tempo reale
 - Operazioni complesse da remoto (es. prenotazioni, referti, consulti)

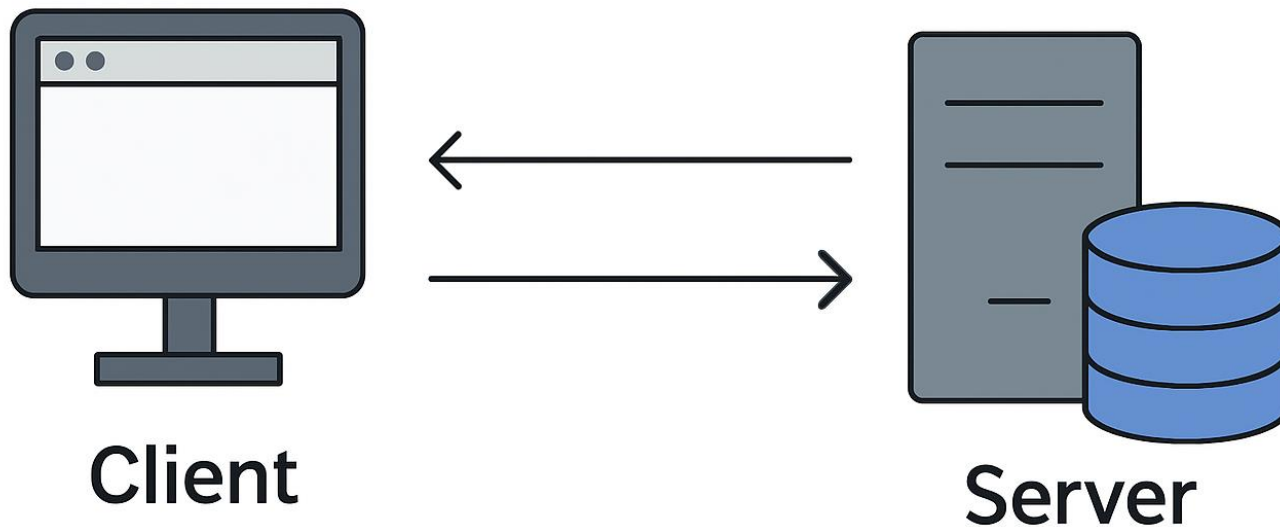
Cuore architetturale: **modello Client-Server**

HTTP: il ponte della comunicazione

- Protocollo che regola lo scambio di dati
- Ogni azione (prenotazioni, consulti, download referti) è una richiesta HTTP

Modello client-server

- Modello di progettazione software che suddivide un'applicazione in due parti principali: il lato client e il lato server
- Il client fa le richieste il server risponde



Client e Server



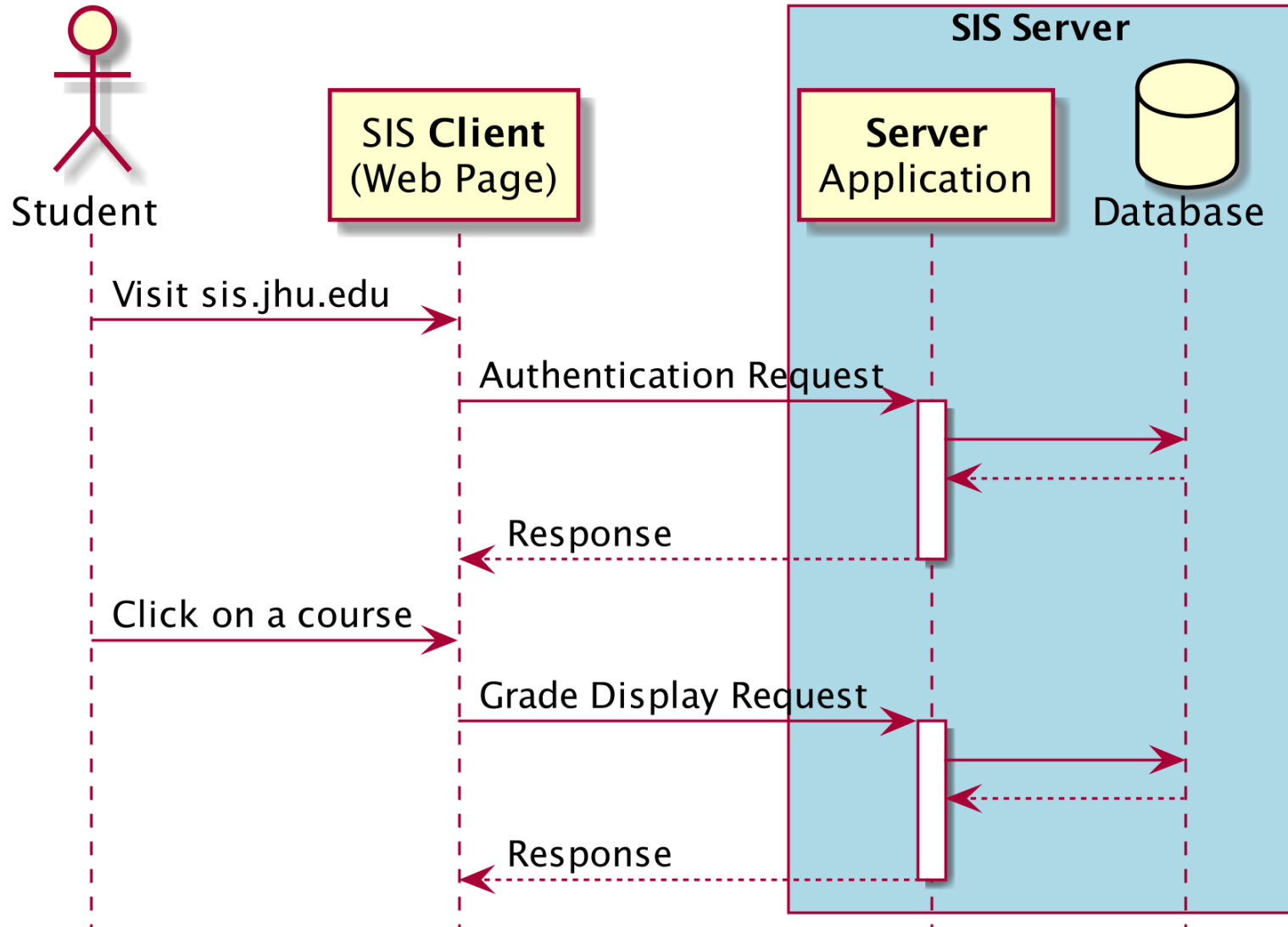
- **Client-Side (Client):**
 - Applicazione eseguita sul computer dell'utente finale.
 - Fornisce l'interfaccia utente (UI) che gestisce l'aspetto e l'interazione con l'utente.
 - Può utilizzare risorse locali del dispositivo, come memoria temporanea e archiviazione locale.
- **Server-Side (Server):**
 - Applicazione che riceve le richieste dai client e contiene la logica per inviare i dati appropriati in risposta.
 - Di solito espone un'interfaccia di programmazione (API) anziché un'interfaccia utente.
 - Spesso include un database per memorizzare persistentemente tutti i dati dell'applicazione.



Esempio interazioni

- Ruoli delle entità:
 - Client: invia richieste, solitamente un browser o un'app
 - Server: elabora le richieste e restituisce risposte
- Esempio pratico:
 1. Il paziente accede al portale sanitario
 2. Il client (browser) invia una richiesta al server
 3. Il server recupera i dati e restituisce la risposta
 4. Il browser visualizza i risultati
- Questo modello è alla base di quasi tutte le applicazioni web in ambito sanitario: FSE, prenotazioni online, referti, ecc.

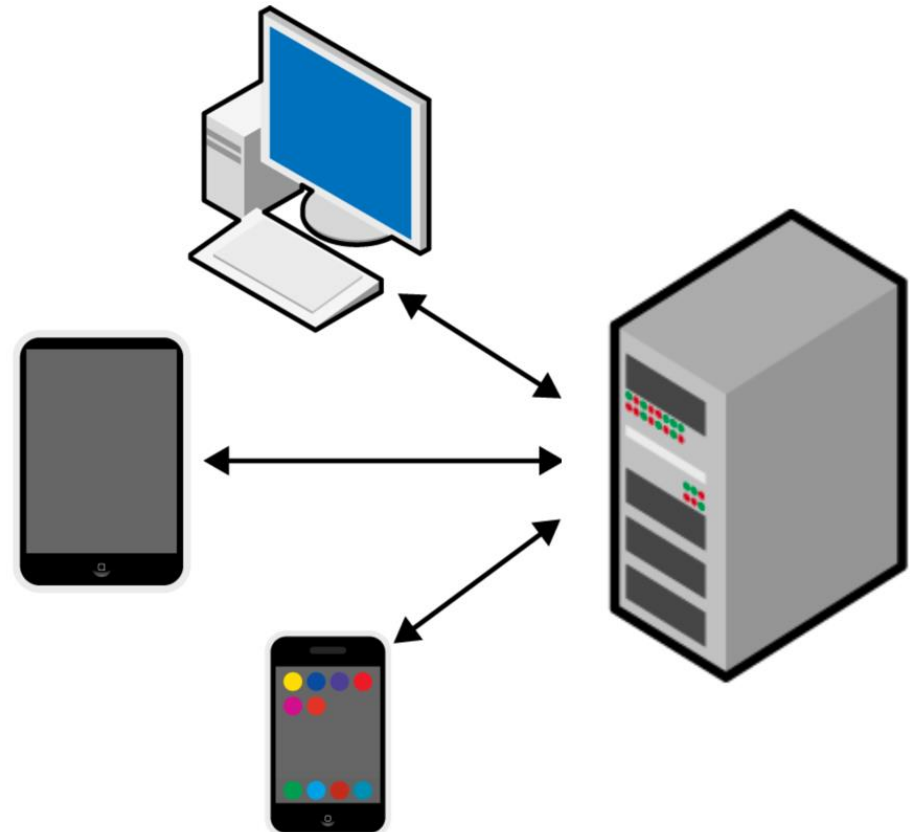
Il Ciclo di Comunicazione



Flessibilità e Centralizzazione



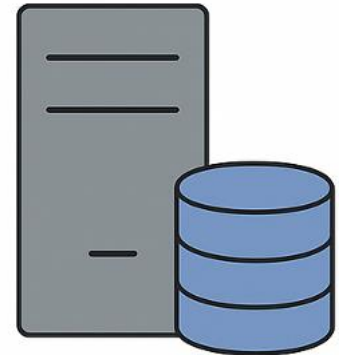
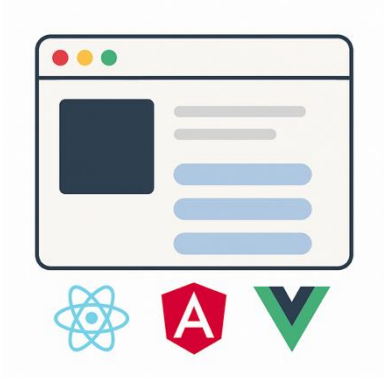
- Centralità del Server
 - Il server funge da punto centrale di elaborazione, memorizzazione e logica applicativa
 - I client possono interagire tramite in server
 - in modo sincrono (in tempo reale)
 - asincrono (in momenti diversi), mant
- Diversità dei Client
 - I client possono essere eterogenei:
 - Web browser
 - App mobile
 - Software desktop
 - Dispositivi IoT
- L'interfaccia può essere adattata a ciascuna piattaforma



Componenti software



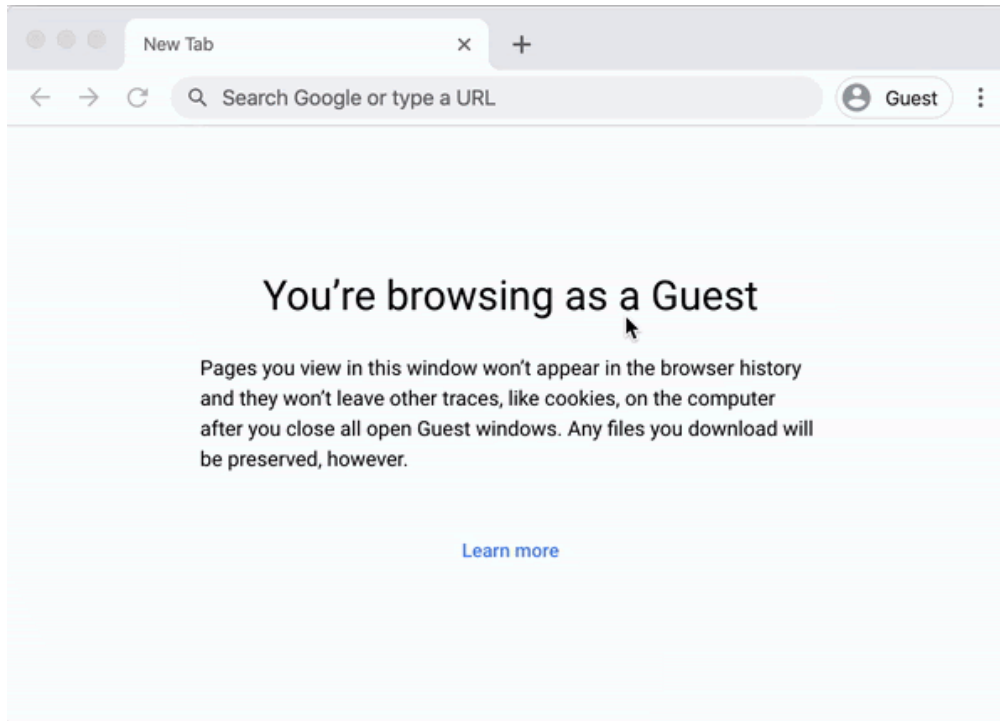
- In una Web App, il codice è suddiviso in:
- Lato Client (Frontend):
 - Codice eseguito sul dispositivo dell'utente
 - Gestisce interfaccia e interazioni
 - Usa JavaScript e framework (React, Angular, Vue)
- Lato Server (Backend):
 - Codice eseguito sul server remoto
 - Gestisce dati e logica dell'applicazione





Navigare nel Web

Cosa è realmente?



1. Digitiamo l'indirizzo
 2. Premiamo invio
 3. Il sito viene caricato!
- Cosa succede tra 2 e 3?

Il Percorso di una Richiesta

- Inserimento dell'indirizzo Web
- Risoluzione del dominio e individuazione del server
- Invio della richiesta al server
 - il browser apre una connessione con il server e invia una richiesta HTTP
 - Il server riceve la richiesta e recupera il file desiderato
- Risposta del server e rendering della pagina
 - Il server invia una risposta HTTP con il contenuto richiesto (HTML, immagini, CSS, script, ecc.)
- Il browser interpreta il codice HTML, e “renderizza” la pagina, rendendola navigabile per l'utente.



Server





Le pagine sono sparse per il mondo





Risorse web

- I contenuti che visualizziamo **non sono nei nostri dispositivi**
- Sono **ospitati su server Web**
 - computer dedicati che archiviano e distribuiscono risorse digitali
- Due tipi di risorse:
 - File static
 - Contenuti **sempre uguali** per tutti gli utenti
 - Contenuti Dinamici
 - Generati **in tempo reale** al momento della richiesta
 - Personalizzati in base all'utente, database o interazione
- Le WebApp moderne combinano **risorse statiche e dinamiche**



Uniform Resource Identifier - URI

- URI (Uniform Resource Identifier)
 - È un **identificatore** univoco per una risorsa sul Web
 - Come un indirizzo postale: dice cosa cercare e dove trovarlo
 - Può essere usato per immagini, pagine, file...
- 1. `mailto:info@ospedalesanita.it`
 - Indica un indirizzo email
- 2. `urn:isbn:978-88-123-4567-0`
 - Identifica un libro tramite ISBN
- 3. `ftp://ftp.esempio.it/immagini/banner.png`
 - Risorsa accessibile via protocollo FTP
- 4. `http://www.laboratoriomedico.it/referti/12345.pdf`
 - 1. → Può essere anche un URL, ma usato come identificatore

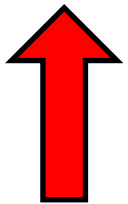


Uniform Resource Locator (URL)

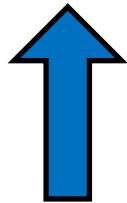
- È il tipo più comune di URI
- Specifica dove si trova la risorsa e come accedervi
- Un identificatore per la posizione di un documento

<https://internet.uniroma2.it/esame/programmazione-web/>

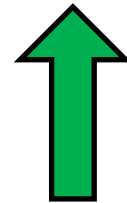
<https://ing.uniroma2.it/area-studenti/guida-dello-studente/>



protocollo



host



path



Domain Name System

Pubblicità ...





Domain Name System - DNS

- Cosa fa il DNS?
 - I computer non capiscono i nomi di dominio (es. `www.google.com`)
 - Comunicano solo tramite indirizzi IP (es. `173.194.34.87`)
 - Il DNS traduce i nomi in indirizzi IP → permette ai dispositivi di comunicare
 - 💡 Analogia: come una rubrica telefonica che collega un nome (contatto) al suo numero
- Noto l'indirizzo il client può aprire un socket verso il server!!

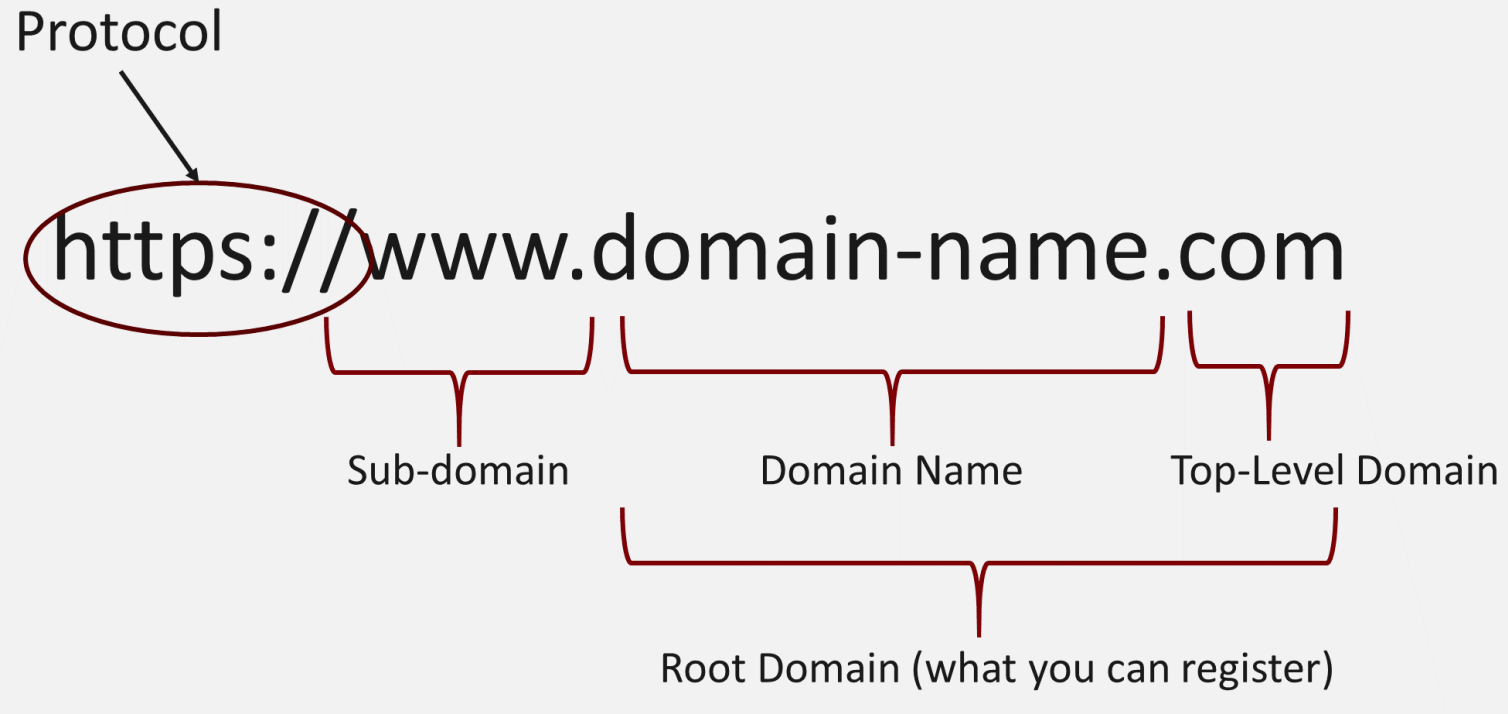


Come funziona la risoluzione DNS

- 🧑💻 L'utente digita un indirizzo nel browser (es. www.google.com)
- 📡 Il computer invia la richiesta a un server DNS
- 🗄️ Il server cerca nel suo database di corrispondenze e restituisce l'indirizzo IP
- 🌐 Il browser usa l'IP per connettersi al server web e caricare il sito



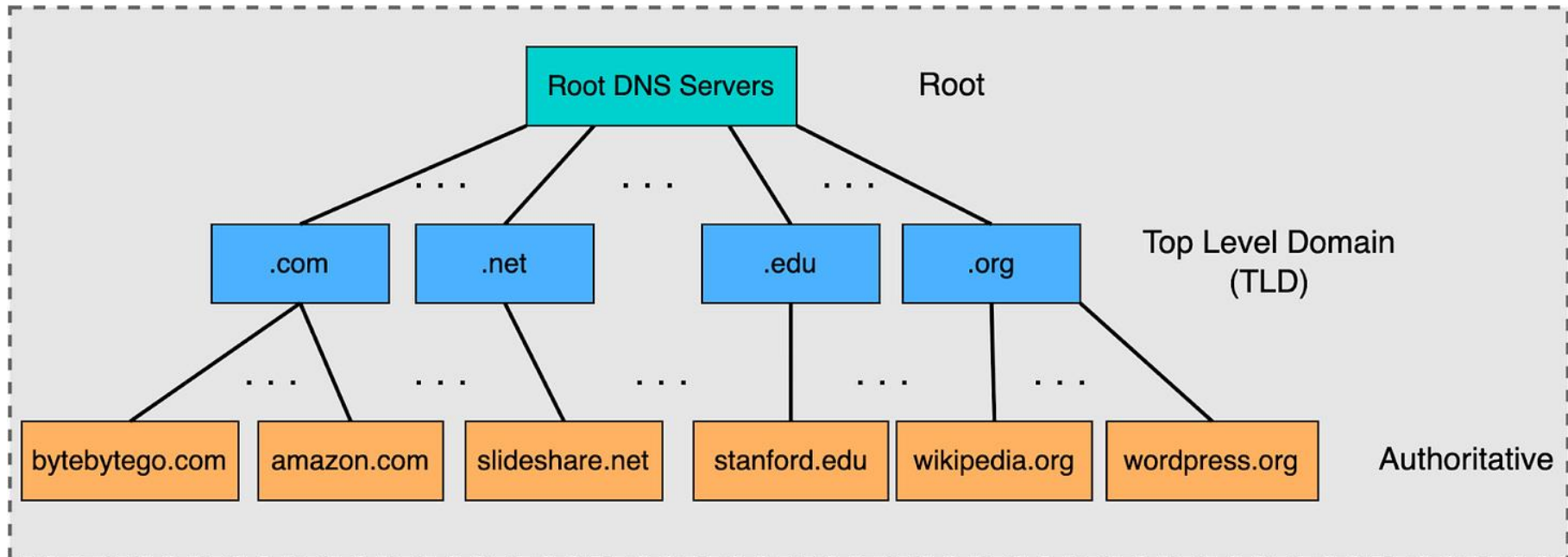
Struttura dei nomi





La struttura gerarchica del DNS

- Gerarchia ad albero, con la radice (.) al vertice
- TLD: .com, .org, .it, ecc.
- Domini secondari e sottodomini





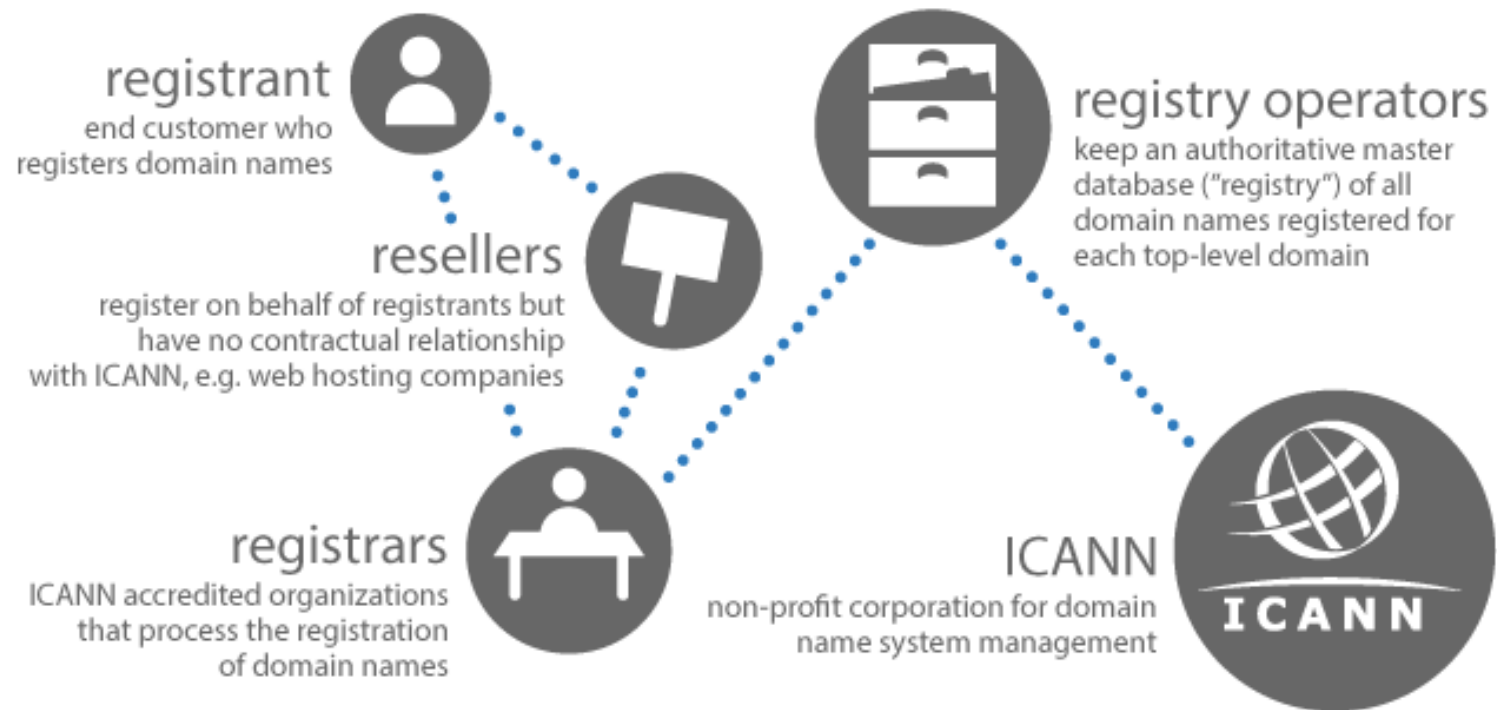
Top-Level Domains, TLD

- Domini di primo livello nazionali
 - Identificano specifici paesi o territori.
 - Sono composti da due lettere, conformemente ai codici ISO 3166.
 - Esempi: .it per l'Italia, .fr per la Francia, .jp per il Giappone.
- Domini di primo livello generici
 - Non sono associati a una specifica nazione.
 - Utilizzati per rappresentare categorie o settori specifici.
 - Esempi comuni:
 - .com: originariamente destinato a entità commerciali.
 - .org: inizialmente riservato a organizzazioni non profit.
 - .net: pensato per infrastrutture di rete.
- Domini di primo livello infrastrutturali
 - Utilizzati per scopi tecnici legati all'infrastruttura di Internet.
 - Attualmente, l'unico TLD di questa categoria è .arpa, impiegato per la risoluzione inversa degli indirizzi IP.

Chi gestisce i nomi di dominio?



- **Registrar:** enti accreditati per registrare domini
- **Registry:** gestisce i TLD e aggiorna i TLD server





Dominio miosito.it

1. TLD (Top-Level Domain): **.it**

- Indica che il dominio appartiene alla zona geografica italiana.
- Ogni TLD ha un gestore ufficiale, detto Registry.

2. Registry: **registro.it**

- È l'organismo che gestisce il TLD .it.
- È operato dal **CNR (Consiglio Nazionale delle Ricerche)**, attraverso l'Istituto di Informatica e Telematica di Pisa.
- Tiene traccia di tutti i domini .it registrati.

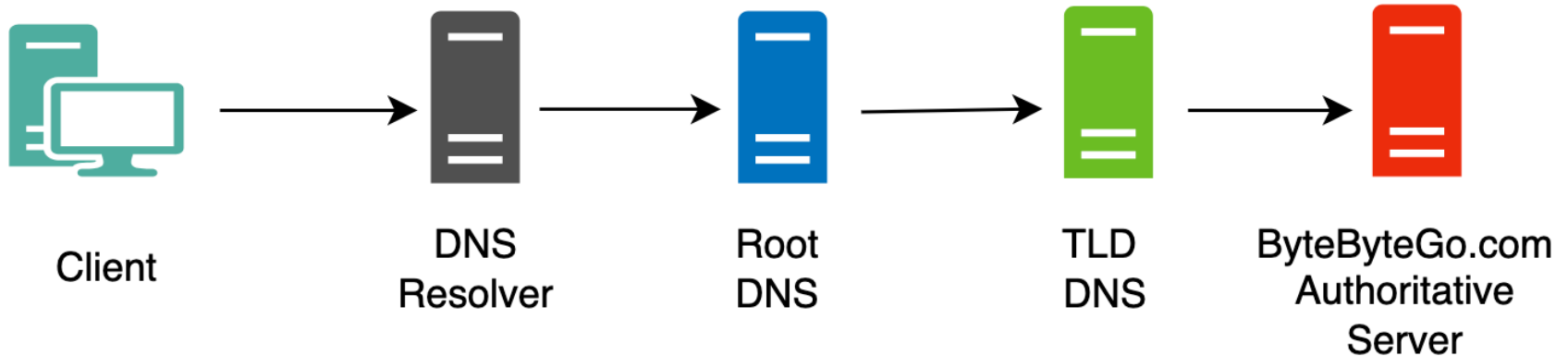
3. Registrar: **aruba, register.it, OVH, ecc.**

- Sono aziende accreditate dal Registro.it.
- Permettono all'utente di registrare un dominio .it.
- L'utente sceglie il dominio (es. miosito.it) e fornisce le informazioni necessarie.

DNS: un sistema distribuito globale



- Distribuito per scalabilità e resilienza
- Root server → TLD server → server autoritativi
- Ogni livello conosce solo il successivo
- Vantaggio: affidabilità e performance



Query miosito.it



1. Il browser controlla la cache
 - Se ha già l'indirizzo IP salvato, lo usa subito.
 - Altrimenti, chiede al resolver
2. Il resolver interroga i Root Server
 - Chiede: “Chi gestisce .it?”
 - I Root Server rispondono: “Vai ai TLD server per .it”.
3. Il resolver interroga i TLD Server di .it
 - Chiede: “Chi è il server autoritativo per miosito.it?”
 - Il TLD server risponde con l'indirizzo dei server autoritativi configurati dal registrar (es. ns1.aruba.it).
4. Il resolver interroga il server autoritativo
 - Chiede: “Qual è l'IP di www.miosito.it?”
 - Il server autoritativo risponde: “L'IP è 93.184.216.34” (esempio fittizio).
5. Il resolver restituisce l'IP al browser
 - Il browser ora sa dove inviare la richiesta HTTP.

Tutti i passaggi possono essere memorizzati (cache) per un tempo determinato (TTL) per evitare richieste ripetute.



Esercizi con il dns

- `nslookup www.google.com`
- `dig www.miosito.it`
- <https://dnschecker.org/>



WWW

Internet



- Internet è una sistema di dispositivi elettronici interconnessi
 - IP: Internet Protocol
- Ogni computer ha un indirizzo IP
 - Es: 160.80.180.186
 - WhatsMyIP.org
- Chi è connesso alla rete può **condividere informazioni**

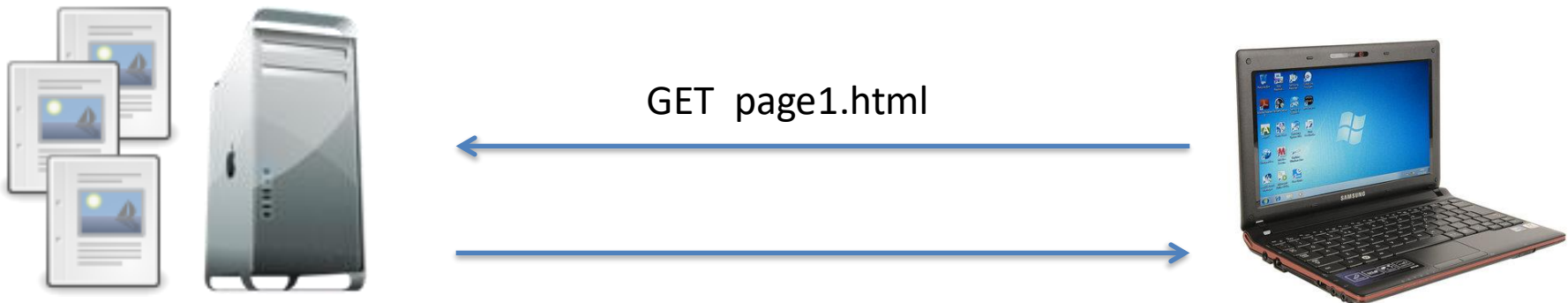


- Il **World Wide Web** (WWW) è uno dei modi con cui le **informazioni** possono essere condivise
- il WWW è un insieme di pagine web ed altri documenti collegati tra loro
 - usando i collegamenti ipertestuali (link)
- Il protocollo principale per scambiare documenti è HTTP
 - HyperText Transfer Protocol
 - lo scrivete spesso quando navigate

Client Web



- I software che fanno **richieste HTTP** ai server si chiamano **client**
 - sono clienti dei server
 - I più comuni client web sono i **Browser**
- Richiesta di un documento
 - L'utente inserisce un indirizzo
 - Il browser chiede un documento al server
 - Il server glielo spedisce
 - Il browser lo mostra all'utente





I Browser

- I browser parlano HTTP come i server web
 - Sia le richieste che le risposte sono gestite mediante HTTP.
- Richiedono documenti di qualsiasi tipo: testo, immagini, video, audio, script, etc.
- Browser famosi
 - Chrome, Internet Explorer/Edge, Firefox, Safari, Opera
- I browser visualizzano i documenti richiesti al server in un finestra
 - spesso gli stessi documenti sono mostrati in modo diverso dai diversi browser

Server web



- Computer che **rispondono a delle richieste di documenti**
 - In realtà chi offre il servizio è un programma: **il server web**
 - Aspetta una richiesta, cerca un documento e lo invia al richiedente
 - Il server web deve parlare HTTP, spesso lo chiamiamo **Server HTTP**
 - I computer server non sono particolari, ogni computer può diventare un server
- Server (programmi) famosi
 - Apache (40.4%)
 - nginx (31.8%)
 - dati Marzo 2020 - netcraft.com



Esercizio

- <https://classroom.github.com/a/CcqcvjC7>



Esercizio: creare un server Web locale

- Avviare un **server HTTP locale** sulla porta 8000

```
npx http-server
```

- Come accedere al server dal browser
 - Sul browser digitiamo: 127.0.0.1:8000

<https://www.npmjs.com/package/http-server>



Serve root e file path

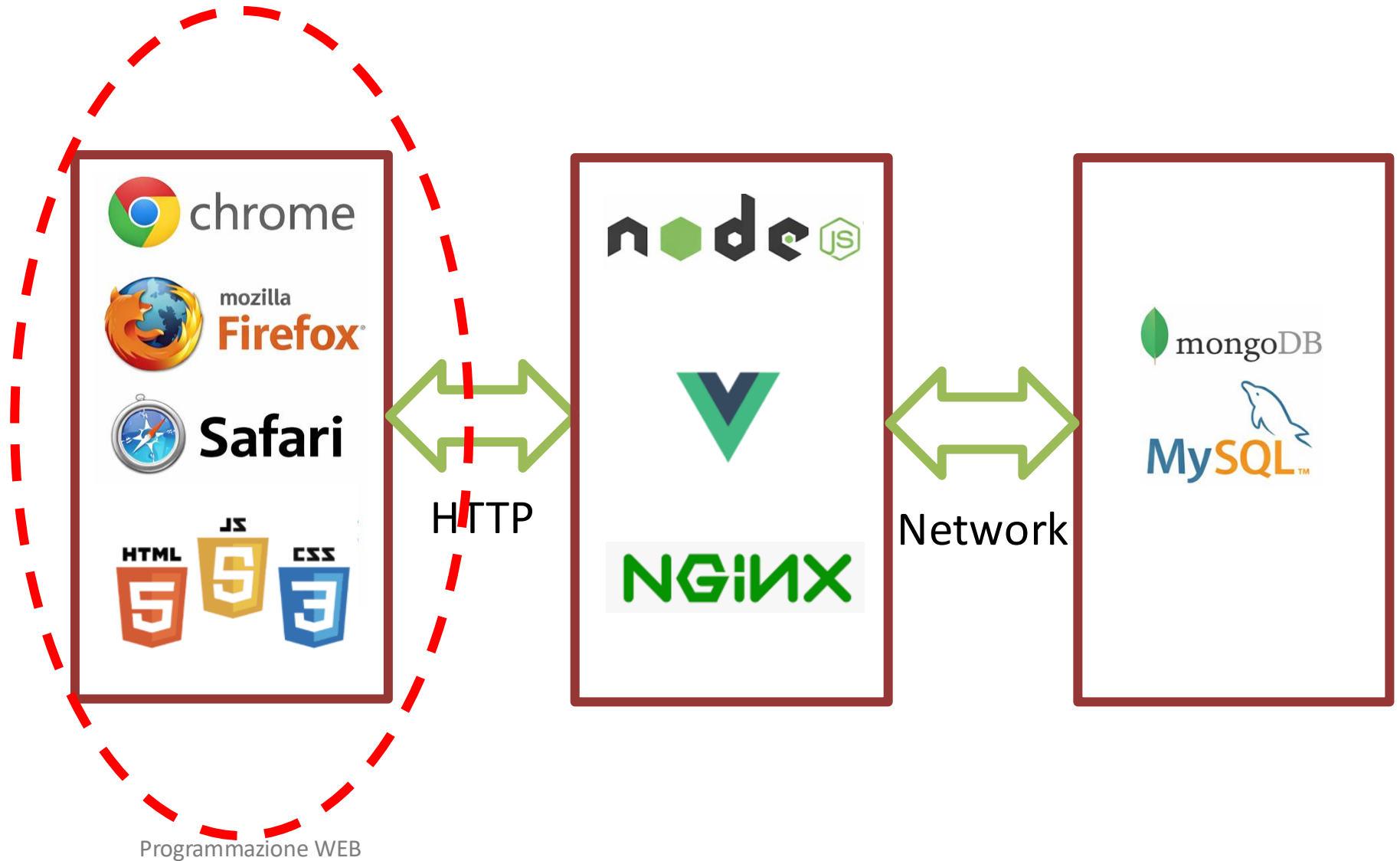
- Root del server
 -  cartella principale del server Web.
 -  punto di partenza da cui vengono servite tutte le risorse (file, pagine).
 -  Quando digitiamo un URL senza percorso (es. www.miosito.it), il browser cerca nella root il file di default
 - `index.html`
- Path
 -  Il path (percorso) indica la posizione di una risorsa all'interno della struttura del server
 - Relativamente alla root
 -  Ogni livello corrisponde a una sottocartella





Pagine web

Full Stack Web Architecture



Primo sito web

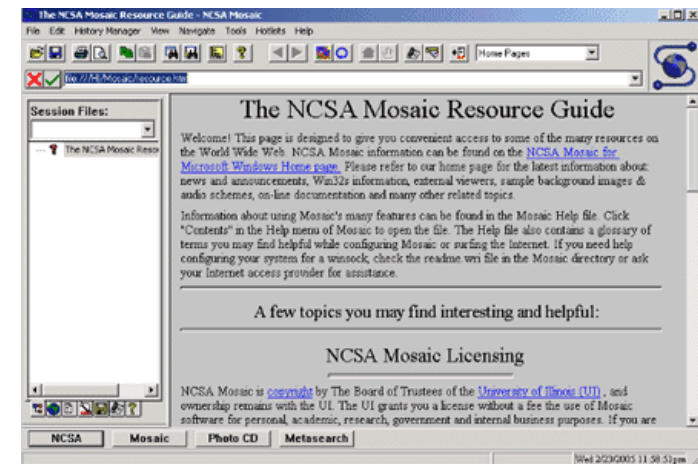


6 agosto 1991

Nascita del web



- Il web nasce nel 1989 a Ginevra, in Svizzera nei laboratori del CERN
- Tim Berners-Lee fu il primo a proporre un sistema di gestione delle informazioni basato sull'ipertesto per collegare i documenti fra loro
 - Lui e Robert Cailliau hanno creato un prototipo
 - Crea il primo sito web 1991
- 1992
 - i server web erano 50.
 - nasce NCSA Mosaic, il primo Browser grafico e il Web iniziò ad essere "di massa"
- *Per approfondire:*
 - *W3C's History Archives*
 - *www.w3.org/History.html*



Gestione di Internet



- Internet Engineering Task Force ([IETF](#)):

- internet protocol standards



- Internet Corporation for Assigned Names and Numbers ([ICANN](#)):

- decides top-level [domain names](#)



- World Wide Web Consortium ([W3C](#)):

- web standards

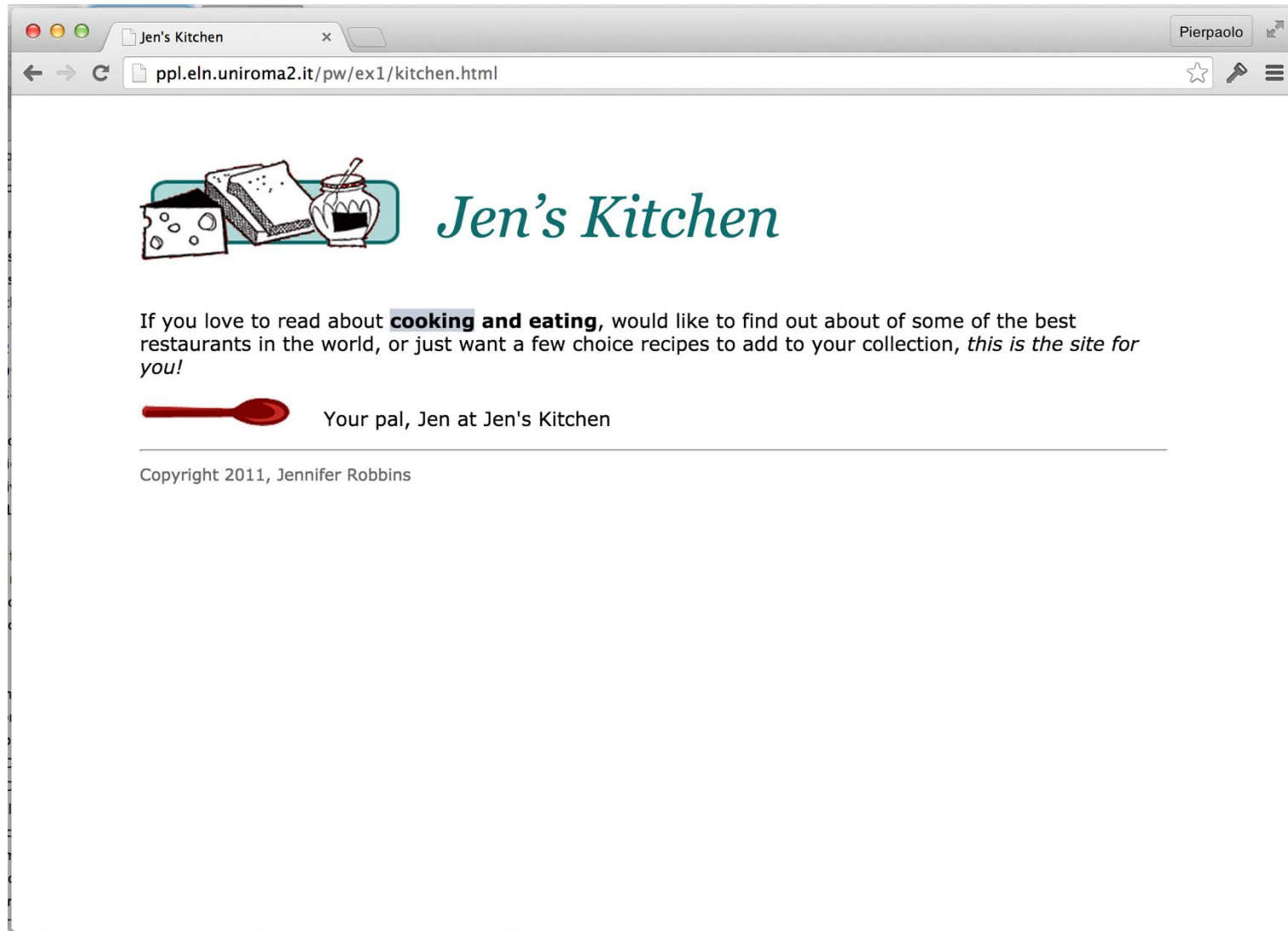


Cosa c'è dentro una pagina web



- **Hypertext Markup Language (HTML):** markup semantico per il contenuto delle pagine web
- **Cascading Style Sheets (CSS):** stile delle pagine web
- **Javascript:** aggiungere interattività alle pagine web

Esempio pagina web



Il file html



- Il file sorgente

<http://www.pw.netgroup.uniroma2.it/ex1/kitchen.html>

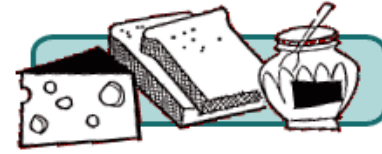
```
1  <!DOCTYPE html>
2  <html>
3  <head>
4  <title>Jen's Kitchen</title>
5  <link rel="stylesheet" href="kitchen.css">
6  </head>
7
8  <body>
9  <h1> Jen's Kitchen</h1>
10
11 <p>If you love to read about <strong>cooking and eating</strong>, would like to
12 find out about of some of the best restaurants in the world, or just want a few
13 choice recipes to add to your collection, <em>this is the site for you!</em></p>
14
15 <p> Your pal, Jen at Jen's
16 Kitchen</p>
17
18 <hr>
19
20 <small>Copyright 2011, Jennifer Robbins</small>
21 </body>
22 </html>
```

Markup



- Il documento è pieno di tag HTML
 - `<!DOCTYPE html>`, `<html>`, `<head>`, `<title>`
 - testo fra parentesi acute
- I tag:
 - non sono mostrati dal browser
 - sono definiti dallo standard
- Molti tag sono a coppie
 - `<body> ... </body>`
 - tag di apertura e tag di chiusura
- Alcuni tag non hanno chiusura
 - `<hr>`, `<img ... >`

Dove sono le figure?



- Ogni figura è in un file separato
 - Alcuni elementi dicono al browser di richiedere altri file
- Esempio
 - Quando il browser trova l'elemento **img**, questo fa una **nuova richiesta http** per l'immagine e quindi la posiziona nella pagina
- Per i video, audio, etc. è più o meno la stessa cosa
- Su connessioni lente si vede comparire l'immagine in un secondo momento





Aggiungere lo stile

- Lo stile è definito in un altro file: *kitchen.css*
 - I file css contengono delle direttive sull'aspetto della pagina

```
1 body { font: normal 1em Verdana; margin: 1em 10%;}  
2 h1 { font: italic 3em Georgia; color: rgb(23, 109, 109); margin: 1em 0 1em;}  
3 img { margin: 0 20px 0 0; }  
4 h1 img { margin-bottom: -20px; }  
5 small { color: #666666;}
```

- Queste istruzioni seguono il linguaggio Cascading Style Sheets

Senza css



Con css

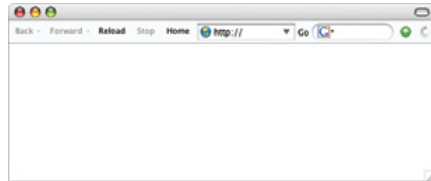


Scambio file



Browser

Server



GET kitchen.html



kitchen.html



GET kitchen.css

GET food.gif

GET spoon.gif



kitchen.css



food.gif



spoon.gif



Recap



- Quando parliamo di pagine web ci si riferisce di solito a ciò che visualizza il browser
 - una insieme contenuti, struttura, stili, immagini, etc.
- Il browser fa diverse richieste HTTP per collezionare le diverse parti
- Quello che viene visualizzato dipende da un documento di struttura HTML, da direttive di stile CSS e da contenuti multimediali





HTML

HTML



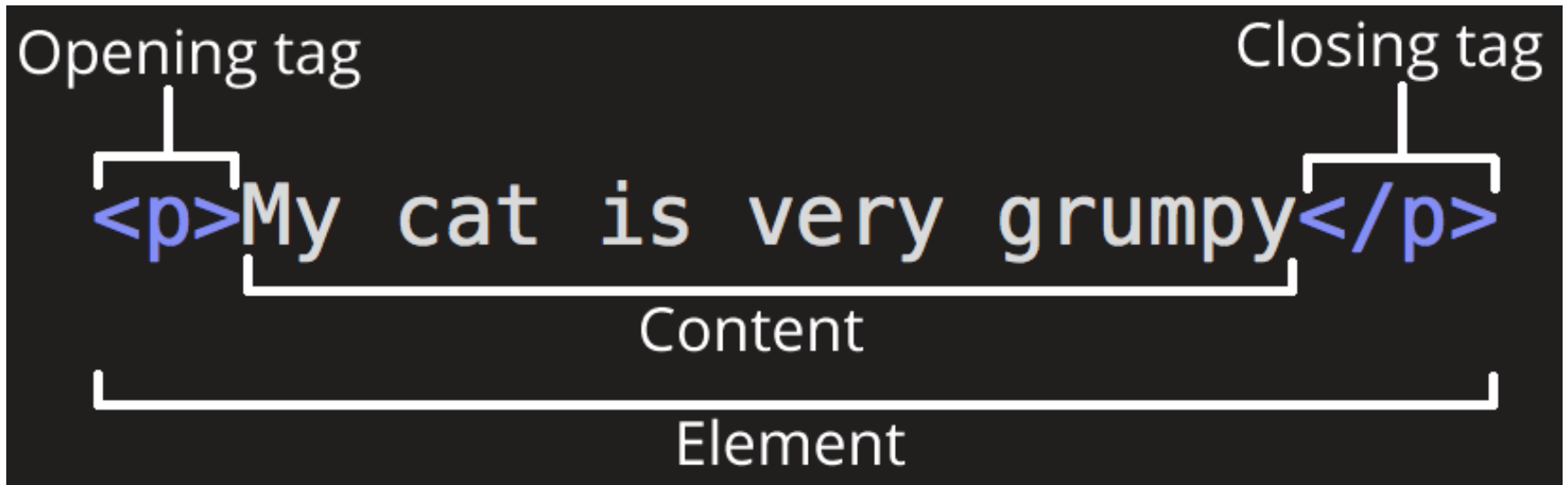
- Sigla per **HyperText Markup Language**
- Linguaggio per **strutturare** documenti ipertestuali
 - Esempio:
 - Documento → curriculum
 - Documento ipertestuale → il curriculum contiene dei [link](#)
 - Documento ipertestuale strutturato → il curriculum ha un titolo
- E' uno standard del W3C
- E' lo standard più utilizzato per la creazione di pagine web visualizzabili attraverso i browser



MARKUP

https://www.w3schools.com/html/html_elements.asp

https://www.w3schools.com/html/html_attributes.asp



https://developer.mozilla.org/it/docs/Learn/HTML/Introduction_to_HTML/Getting_started

Elementi e tag



- I tag possono essere annidati

`<p>This is correct. </p>`



`<p>This is correct. </i></p>`



- Annidamenti scorretti

`<p>This is very wrong! </strong></p>`



Attributi



Attribute

```
<p class="editor-note">My cat is very grumpy</p>
```

- Alcuni elementi hanno attributi
 - coppie chiave-valore definite nel tag di apertura
 - sono specificati dallo standard per i diversi tag
 - forniscono informazioni aggiuntive sull'elemento
- ``
 - chiave: `src`
 - valore: `food.gif`



Raccomandazioni

- Usare Virgolette

```
<a href=https://www.w3schools.com>
```

```
<a href="https://www.w3schools.com">
```

- Necessarie se il valore contiene spazi

```
<p title>About W3Schools>
```



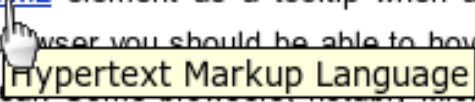
Virgolette doppie e singole

```
<p title='John "ShotGun" Nelson'>
```

```
<p title="John 'ShotGun' Nelson">
```

Se il valore dell'attributo contiene gli apici doppi uso i singoli e viceversa

- L'attributo **title** crea una "tooltip"

Demonstrations of tooltip usage are prevalent on Web pages. Many graphical [Web browsers](#) display the `title` attribute of an [HTML](#) element as a tooltip when a user hovers the mouse cursor over that element; in such a browser you should be able to hover over Wikipedia images and hyperlinks and see a tooltip appear.  [Hypertext Markup Language](#) Microsoft's [Internet Explorer](#),



Elementi vuoti

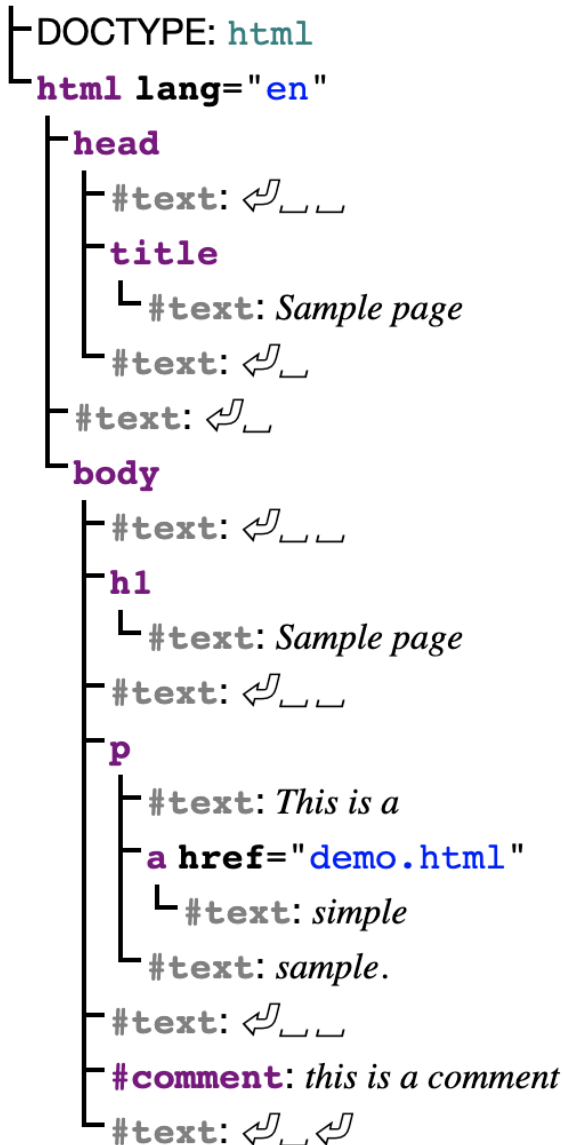
- Sono definiti con un solo tag

`<p>This is a
 paragraph with a
line break.</p>`

- Possono contenere attributi

``

Albero degli elementi



```
<!DOCTYPE html>
<html lang="en">
  <head>
    <title>Sample page</title>
  </head>
  <body>
    <h1>Sample page</h1>
    <p>This is a <a href="demo.html">simple</a> sample.</p>
    <!-- this is a comment -->
  </body>
</html>
```



Recap

- Il markup definisce elementi mediante tag
- Gli elementi posso essere annidati
- Posso fornire informazioni aggizionali su un elemento definendo degli attributi
- Gli elementi formano un albero





Struttura DI DOCUMENTI HTML



Head, Body

```
<!DOCTYPE html>
<html>
  <head>
    <title>Titolo della Pagina</title>
  </head>
  <body>
    <h1>Questo è il Body della Pagina</h1>
    <p>Gli elementi contenuti nel body vengono
mostrati dal browser.</p>
  </body>
</html>
```

L'elemento **<html>** è la radice dell'albero e contiene tutti gli altri

L'elemento **<head>** fornisce informazioni circa il documento

L'elemento **<body>** contiene il contenuto visibile



Dichiarazione `<!DOCTYPE>`

- La dichiarazione `<!DOCTYPE>` serve a dire al browser il tipo di documento HTML
 - ci sono diversi tipi di documento
- Il browser deve sapere il tipo e la versione dell'HTML da visualizzare
- La dichiarazione non è case sensitive:
 - `<!DOCTYPE html>`
 - `<!DOCTYPE HTML>`
 - `<!doctype html>`
 - `<!Doctype Html>`



Dichiarazioni comuni

- HTML5

```
<!DOCTYPE html>
```

- HTML 4.01

```
<!DOCTYPE HTML PUBLIC "-//W3C//DTD HTML 4.01  
Transitional//EN" "http://www.w3.org/TR/html4/loose.dtd">
```

- XHTML 1.0

```
<!DOCTYPE html PUBLIC "-//W3C//DTD XHTML 1.0 Transitional//EN"  
"http://www.w3.org/TR/xhtml1/DTD/xhtml1-transitional.dtd">
```

Evoluzione HTML e CSS



- Versioni dell'HTML

HTML	1991
HTML+	1993
HTML 2.0	1995
HTML 3.2	1997
HTML 4.01	1999
XHTML	2000
HTML5	2012 (2014)

- Versioni CSS

CSS1	1996
CSS2	2004
CSS2.1	2011
CSS3	2014



Sezione <head>

```
<head>  
  <meta charset="utf-8">  
  <link rel="stylesheet" href="style.css">  
  <title>La mia prima pagina</title>  
</head>
```

- Contiene metadati relativi alla pagina
 - descrizione, charset, autore, etc
- Generalmente il contenuto contenuto all'interno di questo tag non viene visualizzato nella pagina web
- Contiene il titolo ...



Titolo della Pagina: <title>

- Definisce il titolo della pagina
 - visualizzato dal browser nella toolbar
 - è usato quando la pagina è memorizzata nei preferiti
 - è usato dai motori di ricerca
- Tale elemento è obbligatorio nelle pagine
 - il documento non è un doc HTML valido
- Ci può essere un solo elemento <title>

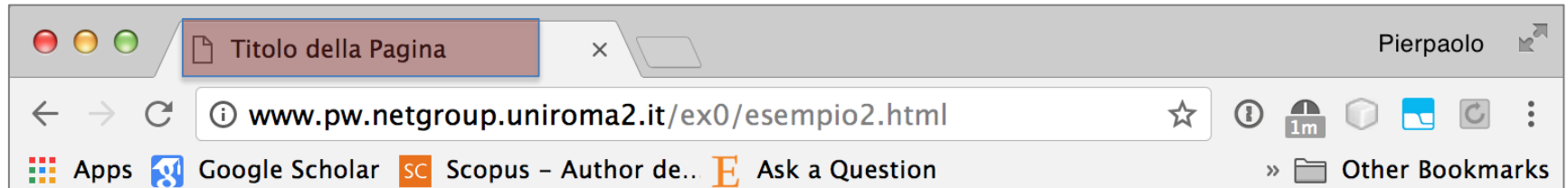


Sezione <body>

```
<body>
  <header>
    <!-- Intestazione del corpo della pagina web
         (ad es. logo, barra di navigazione) -->
  </header>
  <main>
    <!-- Sezione principale del corpo della pagina web
         (dove si trova la maggior parte del contenuto) -->
  </main>
  <footer>
    <!-- Piè di pagina del corpo della pagina web
         (ad es. informazioni sul copyright) -->
  </footer>
</body>
```

- Contiene la parte visibile della pagina
- Normalmente diviso in sezioni

Visualizzazione



Questo il Body della Pagina

Gli elementi contenuti nel body vengono mostrati dal browser.